

ĐẠI HỌC PHENIKAA
TRƯỜNG CÔNG NGHỆ THÔNG TIN



BÁO CÁO: ỨNG DỤNG PHÂN TÁN

***Đề tài: Nghiên cứu và mở rộng hệ thống truyền thông
điện phân tán NSQ***

Giảng viên hướng dẫn: TS. Nguyễn Lệ Thu.

Thành viên nhóm:

1. Nguyễn Thị Thùy Linh – 23010633.
2. Nguyễn Anh Quân – 23010375.

Lớp: CSE702063-1-3-25(N03).

Hà Nội, 2026.

MỤC LỤC

1. DỰ ÁN ĐÃ CHỌN	1
1.1 Giới thiệu dự án	1
1.2 Kiến trúc hệ thống	1
1.3 Đặc điểm của hệ thống phân tán NSQ.....	2
2. MỤC ĐÍCH, CHỨC NĂNG VÀ ỨNG DỤNG THỰC TẾ	4
2.1 Mục đích của dự án	4
2.2 Chức năng chính của hệ thống	4
2.3 Ứng dụng thực tế	6
2.3.1. Hệ thống thương mại điện tử.....	6
2.3.2. Hệ thống Microservices.....	6
2.3.3. Hệ thống xử lý Log.....	6
2.3.4. Hệ thống Internet of Things (IoT)	6
2.3.5. Hệ thống xử lý dữ liệu thời gian thực	7
2.4 Đánh giá.....	7
3. THIẾT LẬP CÀI ĐẶT VÀ KẾT QUẢ THỰC NGHIỆM	8
3.1 Môi trường thực nghiệm.....	8
3.2 Tải và biên dịch mã nguồn	8
3.3 Khởi động hệ thống NSQ	9
3.4 Thực nghiệm cân bằng tải (Load Balancing)	9
3.5 Thực nghiệm khả năng chịu lỗi (Fault Tolerance)	10
3.6 Đánh giá kết quả thực nghiệm.....	11
4. PHÁT TRIỂN TÍNH NĂNG MỚI	12
4.1. Priority-Based Topic Classification and Monitoring.....	12
4.1.1. Mục đích	12
4.1.2. Thiết kế	12
4.1.3. Cài đặt.....	12
4.1.4. Kết quả.....	13
4.1.5. Liên hệ với hệ phân tán	13

4.1.6. Đánh giá.....	13
4.2. Consumer Health Monitoring.....	14
4.2.1. Mục đích.....	14
4.2.2. Thiết kế.....	14
4.2.3. Cài đặt.....	14
4.2.4. Kết quả.....	14
4.2.5. Kiểm thử.....	15
4.2.6. Liên hệ với hệ phân tán	15
4.2.7. Đánh giá.....	15
4.3 Tổng kết các tính năng mở rộng.....	15
5. ĐÓNG GÓP CỦA THÀNH VIÊN	17
5.1 Phân công công việc.....	17
5.2 Kết quả làm việc nhóm.....	17
KẾT LUẬN	18

1. DỰ ÁN ĐÃ CHỌN

1.1 Giới thiệu dự án

Trong các hệ thống phần mềm hiện đại, việc trao đổi dữ liệu giữa các thành phần của hệ thống đóng vai trò rất quan trọng. Khi số lượng người dùng và khối lượng dữ liệu ngày càng tăng, các hệ thống cần một cơ chế truyền thông điệp hiệu quả nhằm đảm bảo khả năng mở rộng, tính sẵn sàng và khả năng chịu lỗi. Một trong những giải pháp được sử dụng phổ biến hiện nay là hệ thống hàng đợi thông điệp phân tán (Distributed Messaging System).

Sau khi tìm hiểu các dự án trong kho lưu trữ Awesome Distributed System Projects, nhóm lựa chọn nghiên cứu dự án NSQ. Đây là một nền tảng truyền thông điệp phân tán mã nguồn mở được phát triển bởi Bitly và được viết hoàn toàn bằng ngôn ngữ Golang. NSQ được thiết kế nhằm xử lý lượng lớn thông điệp theo thời gian thực, hỗ trợ kiến trúc phân tán và có khả năng mở rộng theo chiều ngang.

Thông tin dự án:

- Tên dự án: NSQ
- Địa chỉ mã nguồn: <https://github.com/nsqio/nsq> .
- Nguồn dự án: Dự án được lựa chọn từ kho lưu trữ Awesome Distributed System Projects theo yêu cầu của học phần Ứng dụng phân tán.
- Ngôn ngữ phát triển: Golang
- Loại hệ thống: Distributed Messaging Platform
- Giấy phép: MIT License

NSQ được sử dụng để kết nối các dịch vụ trong hệ thống phân tán thông qua mô hình Publish/Subscribe. Hệ thống cho phép Producer gửi thông điệp đến các Topic, sau đó các Consumer sẽ nhận và xử lý các thông điệp này một cách độc lập. Điều này giúp giảm sự phụ thuộc giữa các thành phần trong hệ thống và tăng khả năng mở rộng.

1.2 Kiến trúc hệ thống

NSQ được xây dựng theo mô hình truyền thông điệp phân tán (Distributed Messaging System), cho phép các thành phần trong hệ thống giao tiếp với nhau thông qua cơ chế Publish/Subscribe. Kiến trúc của NSQ được thiết kế đơn giản nhưng vẫn đảm bảo khả năng mở rộng, hiệu năng cao và khả năng chịu lỗi trong môi trường phân tán.

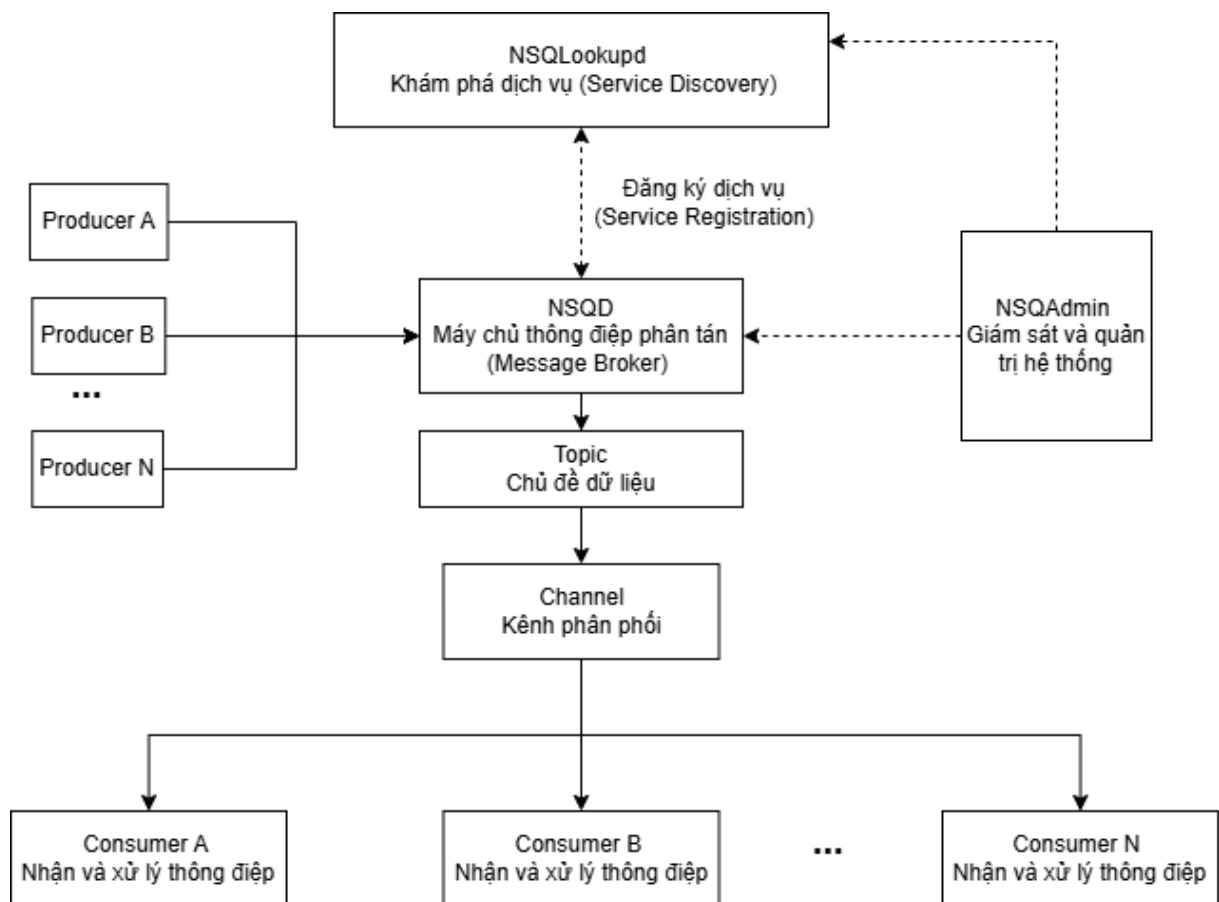
Hệ thống gồm ba thành phần chính:

NSQLookupd: NSQLookupd là thành phần thực hiện chức năng Service Discovery. Thành phần này lưu trữ thông tin về các NSQD đang hoạt động trong hệ thống và hỗ trợ các Consumer tìm kiếm NSQD để kết nối.

NSQD: NSQD là thành phần cốt lõi của hệ thống, chịu trách nhiệm tiếp nhận, lưu trữ và phân phối thông điệp. Đây là nơi các Producer gửi dữ liệu và các Consumer nhận dữ liệu để xử lý.

NSQAdmin: NSQAdmin là giao diện web hỗ trợ giám sát và quản trị hệ thống. Thông qua NSQAdmin, người quản trị có thể theo dõi Topic, Channel, Consumer, lưu lượng thông điệp và các thông số hoạt động của hệ thống theo thời gian thực.

Trong mô hình hoạt động của NSQ, Producer gửi thông điệp đến NSQD. Các thông điệp được lưu trong các Topic và phân phối đến các Consumer thông qua các Channel. NSQLookupd đóng vai trò trung gian giúp các thành phần trong hệ thống tự động tìm thấy nhau mà không cần cấu hình thủ công.



Hình 1. Kiến trúc tổng quan của hệ thống NSQ.

1.3 Đặc điểm của hệ thống phân tán NSQ

NSQ sở hữu nhiều đặc điểm tiêu biểu của một hệ thống phân tán hiện đại:

- Khả năng mở rộng (Scalability): Hệ thống cho phép bổ sung thêm nhiều Producer hoặc Consumer mà không cần thay đổi kiến trúc hiện tại.
- Cân bằng tải (Load Balancing): Thông điệp được phân phối tự động giữa các Consumer trong cùng một Channel nhằm tối ưu tài nguyên xử lý.

- Khả năng chịu lỗi (Fault Tolerance): Khi một Consumer gặp sự cố hoặc ngừng hoạt động, các Consumer còn lại vẫn có thể tiếp tục xử lý thông điệp mà không làm gián đoạn hệ thống.
- Xử lý bất đồng bộ (Asynchronous Processing): Producer và Consumer hoạt động độc lập với nhau, giúp giảm sự phụ thuộc giữa các dịch vụ trong hệ thống.
- Hiệu năng cao: NSQ được phát triển bằng Golang nên có khả năng xử lý lượng lớn thông điệp với độ trễ thấp và hiệu năng cao.
- Khả năng phân tán (Distributed Architecture): Hệ thống có thể triển khai trên nhiều máy chủ khác nhau, các node giao tiếp với nhau thông qua mạng.

Nhờ các đặc điểm trên, NSQ được sử dụng rộng rãi trong các hệ thống Microservices, thương mại điện tử, xử lý log, hệ thống IoT và các ứng dụng yêu cầu xử lý dữ liệu thời gian thực.

Trong phạm vi đồ án, nhóm tập trung thực nghiệm hai đặc tính quan trọng của hệ thống phân tán là khả năng cân bằng tải (Load Balancing) và khả năng chịu lỗi (Fault Tolerance). Đồng thời nhóm phát triển thêm hai chức năng mới trên giao diện NSQAdmin nhằm hỗ trợ giám sát hệ thống gồm Priority Topic Monitoring và Consumer Health Monitoring.

2. MỤC ĐÍCH, CHỨC NĂNG VÀ ỨNG DỤNG THỰC TẾ

2.1 Mục đích của dự án

Trong các hệ thống phần mềm hiện đại, đặc biệt là các hệ thống có quy mô lớn hoặc được xây dựng theo kiến trúc Microservices, việc trao đổi dữ liệu giữa các thành phần là một yêu cầu quan trọng. Nếu các dịch vụ giao tiếp trực tiếp với nhau, hệ thống sẽ dễ gặp các vấn đề như quá tải, phụ thuộc chặt chẽ giữa các thành phần và khó mở rộng.

NSQ được xây dựng nhằm giải quyết các vấn đề trên thông qua cơ chế truyền thông điệp phân tán. Hệ thống đóng vai trò là cầu nối trung gian giữa các dịch vụ, cho phép các thành phần trao đổi dữ liệu một cách bất đồng bộ mà không cần kết nối trực tiếp với nhau.

Mục tiêu chính của NSQ là cung cấp một nền tảng truyền thông điệp phân tán giúp các dịch vụ giao tiếp theo cơ chế bất đồng bộ, giảm sự phụ thuộc giữa các thành phần và hỗ trợ triển khai các hệ thống có khả năng mở rộng cao. Bên cạnh đó, NSQ còn giúp nâng cao khả năng chịu lỗi và tối ưu hiệu năng xử lý dữ liệu trong môi trường thời gian thực.

Nhờ đó, NSQ giúp giảm tải cho các dịch vụ xử lý nghiệp vụ và nâng cao tính ổn định của toàn bộ hệ thống.

2.2 Chức năng chính của hệ thống

NSQ hoạt động dựa trên mô hình Publish/Subscribe, trong đó các thông điệp được gửi từ Producer đến Consumer thông qua Topic và Channel.

Producer: Producer là thành phần chịu trách nhiệm tạo và gửi thông điệp vào hệ thống.

Các chức năng chính:

- Tạo dữ liệu hoặc sự kiện.
- Gửi thông điệp đến Topic.
- Không cần biết Consumer nào sẽ xử lý thông điệp.

Ví dụ: Trong một website thương mại điện tử, khi khách hàng đặt hàng thành công, hệ thống có thể gửi một thông điệp chứa thông tin đơn hàng đến NSQ.

Topic: Topic là nơi tiếp nhận và lưu trữ các thông điệp từ Producer.

Các chức năng chính:

- Nhận thông điệp từ Producer.
- Quản lý luồng dữ liệu.
- Phân phối thông điệp đến các Channel.

Một Topic có thể phục vụ nhiều mục đích khác nhau như:

- order-topic
- payment-topic

- notification-topic

Channel: Channel là thành phần trung gian giữa Topic và Consumer.

Các chức năng chính:

- Nhận thông điệp từ Topic.
- Phân phối thông điệp đến Consumer.
- Thực hiện cân bằng tải giữa các Consumer.

Mỗi Channel hoạt động độc lập và đảm bảo rằng mỗi thông điệp chỉ được xử lý một lần trong cùng một Channel.

Consumer: Consumer là thành phần nhận và xử lý thông điệp.

Các chức năng chính:

- Kết nối đến Channel.
- Nhận thông điệp từ NSQ.
- Thực hiện xử lý nghiệp vụ.
- Xác nhận hoàn thành xử lý thông điệp với NSQ.

Ví dụ: Khi nhận được thông tin đơn hàng mới, Consumer có thể:

- Gửi email xác nhận.
- Cập nhật kho hàng.
- Tạo hóa đơn.

NSQLookupd: NSQLookupd thực hiện chức năng Service Discovery trong hệ thống.

Các chức năng chính:

- Quản lý thông tin các node NSQD.
- Hỗ trợ Consumer tìm kiếm NSQD.
- Tự động cập nhật trạng thái hệ thống.

Nhờ có NSQLookupd, các Consumer không cần biết chính xác địa chỉ của từng NSQD.

NSQAdmin: NSQAdmin là giao diện quản trị của hệ thống.

Các chức năng chính:

- Theo dõi Topic.
- Theo dõi Channel.
- Theo dõi Consumer.
- Hiện thị số lượng thông điệp.
- Giám sát hoạt động hệ thống.

Trong quá trình thực nghiệm, nhóm sử dụng NSQAdmin để quan sát hoạt động của Producer, Consumer và các thông điệp được truyền trong hệ thống. Đồng thời, đây

cũng là nền tảng để nhóm phát triển các tính năng mở rộng phục vụ công tác giám sát và quản trị hệ thống phân tán.

2.3 Ứng dụng thực tế

Hiện nay, các hệ thống truyền thông điệp phân tán được sử dụng rộng rãi trong nhiều lĩnh vực khác nhau.

2.3.1. Hệ thống thương mại điện tử

Khi khách hàng đặt hàng thành công, hệ thống có thể tạo các thông điệp để:

- Xử lý thanh toán.
- Gửi email xác nhận.
- Cập nhật tồn kho.
- Tạo vận đơn giao hàng.

Việc sử dụng hàng đợi thông điệp giúp giảm tải cho hệ thống và tăng tốc độ phản hồi cho người dùng.

2.3.2. Hệ thống Microservices

Trong kiến trúc Microservices, các dịch vụ thường hoạt động độc lập.

NSQ đóng vai trò trung gian giúp:

- Giảm sự phụ thuộc giữa các dịch vụ.
- Tăng khả năng mở rộng.
- Đảm bảo dữ liệu được truyền tải ổn định.

Trong phạm vi đề án, nhóm sử dụng NSQ để mô phỏng quá trình truyền và xử lý thông điệp giữa nhiều Producer và Consumer, qua đó đánh giá khả năng cân bằng tải và khả năng chịu lỗi của hệ thống trong môi trường phân tán.

2.3.3. Hệ thống xử lý Log

Các hệ thống lớn thường sinh ra hàng triệu bản ghi log mỗi ngày.

NSQ có thể được sử dụng để:

- Thu thập log.
- Phân phối log đến các dịch vụ phân tích.
- Giảm tải cho hệ thống ghi log.

2.3.4. Hệ thống Internet of Things (IoT)

Trong các hệ thống IoT, hàng nghìn thiết bị có thể gửi dữ liệu liên tục.

NSQ giúp:

- Thu thập dữ liệu cảm biến.
- Phân phối dữ liệu đến các dịch vụ xử lý.
- Đảm bảo khả năng mở rộng khi số lượng thiết bị tăng lên.

2.3.5. Hệ thống xử lý dữ liệu thời gian thực

Các hệ thống như: Giám sát giao thông , Theo dõi tài chính, Giám sát an ninh, Hệ thống cảnh báo thời gian thực đều có thể sử dụng NSQ để xử lý dữ liệu với độ trễ thấp và hiệu năng cao.

2.4 Đánh giá

Qua quá trình tìm hiểu và thực nghiệm, nhóm nhận thấy NSQ là một nền tảng truyền thông điệp phân tán mạnh mẽ với kiến trúc đơn giản nhưng hiệu quả. Hệ thống hỗ trợ tốt các đặc tính quan trọng của hệ phân tán như cân bằng tải, khả năng chịu lỗi và khả năng mở rộng. Ngoài các chức năng có sẵn, nhóm còn phát triển thêm hai tính năng gồm Priority Topic Monitoring và Consumer Health Monitoring nhằm hỗ trợ việc giám sát và quản trị hệ thống hiệu quả hơn. Các tính năng này được xây dựng dựa trên việc khai thác thông tin từ nhiều thành phần trong hệ thống NSQ, góp phần thể hiện các đặc trưng của hệ thống phân tán như khả năng mở rộng, giám sát tập trung và quản lý nhiều nút xử lý.

3. THIẾT LẬP CÀI ĐẶT VÀ KẾT QUẢ THỰC NGHIỆM

3.1 Môi trường thực nghiệm

Nhóm thực hiện cài đặt và thử nghiệm hệ thống NSQ trên nền tảng GitHub Codespaces nhằm tận dụng môi trường Linux trực tuyến, hỗ trợ đầy đủ Golang và các công cụ phát triển cần thiết cho việc xây dựng và vận hành hệ thống phân tán.

Thành phần	Giá trị
Hệ điều hành	Linux (GitHub Codespaces)
Ngôn ngữ lập trình	Golang
IDE	Visual Studio Code
Quản lý mã nguồn	GitHub
Nền tảng thực nghiệm	GitHub Codespaces
Distributed Messaging Platform	NSQ
Trình duyệt	Google Chrome
Công cụ quản trị	NSQAdmin

Bảng 3.1. Môi trường thực nghiệm

3.2 Tải và biên dịch mã nguồn

Nhóm lựa chọn dự án NSQ từ GitHub và tiến hành tải mã nguồn về môi trường Codespaces.

Lệnh tải mã nguồn: `git clone https://github.com/nsqio/nsq.git`

Sau khi tải mã nguồn, nhóm tiến hành biên dịch dự án bằng Makefile được cung cấp sẵn.

Lệnh build: `make`

```
@ttlinhnguyen205 →/workspaces/nsq-distributed-system (main) $ ls -l build
total 94564
-rwxrwxrwx 1 codespace codespace 9413781 Jun 15 14:21 nsq_stat
-rwxrwxrwx 1 codespace codespace 10014244 Jun 15 14:21 nsq_tail
-rwxrwxrwx 1 codespace codespace 10145843 Jun 15 14:21 nsq_to_file
-rwxrwxrwx 1 codespace codespace 10091226 Jun 15 14:21 nsq_to_http
-rwxrwxrwx 1 codespace codespace 10109840 Jun 15 14:21 nsq_to_nsq
-rwxrwxrwx 1 codespace codespace 15308652 Jun 15 14:21 nsqadmin
-rwxrwxrwx 1 codespace codespace 12482057 Jun 15 14:21 nsqd
-rwxrwxrwx 1 codespace codespace 10612256 Jun 15 14:21 nsqlookupd
-rwxrwxrwx 1 codespace codespace 8635754 Jun 15 14:24 to_nsq
```

Hình 2. Các tệp thực thi được tạo thành công sau quá trình biên dịch.

Kết quả cho thấy toàn bộ thành phần của NSQ được biên dịch thành công mà không phát sinh lỗi.

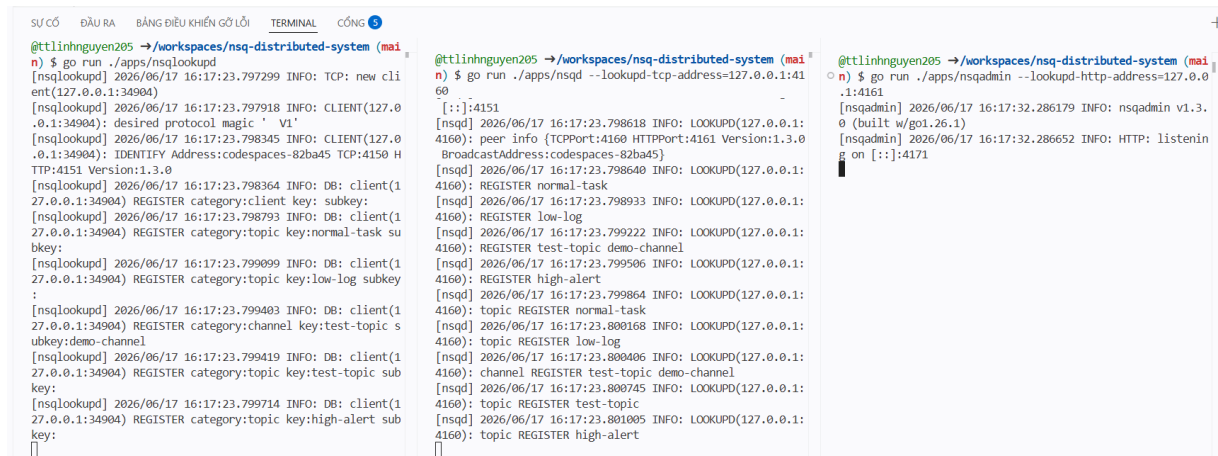
3.3 Khởi động hệ thống NSQ

Sau khi build thành công, nhóm tiến hành khởi động các thành phần của hệ thống theo trình tự sau:

Bước 1: Khởi động NSQLookupd: NSQLookupd thực hiện chức năng Service Discovery, cho phép các thành phần trong hệ thống tự động tìm thấy nhau.

Bước 2: Khởi động NSQD: NSQD là thành phần tiếp nhận, lưu trữ và phân phối thông điệp trong hệ thống.

Bước 3: Khởi động NSQAdmin: NSQAdmin cho phép giám sát Topic, Channel, Consumer và trạng thái hoạt động của hệ thống.



```
@tlinhnguyen205 →/workspaces/nsq-distributed-system (main) $ go run ./apps/nsqllookupd
[nsqllookupd] 2026/06/17 16:17:23.797299 INFO: TCP: new client(127.0.0.1:34904)
[nsqllookupd] 2026/06/17 16:17:23.797918 INFO: CLIENT(127.0.0.1:34904): desired protocol magic ' V1'
[nsqllookupd] 2026/06/17 16:17:23.798345 INFO: CLIENT(127.0.0.1:34904): IDENTIFY Address:codespaces-82ba45 TCP:4150 HTTP:4151 Version:1.3.0
[nsqllookupd] 2026/06/17 16:17:23.798364 INFO: DB: client(127.0.0.1:34904) REGISTER category:client key:subkey:
[nsqllookupd] 2026/06/17 16:17:23.798793 INFO: DB: client(127.0.0.1:34904) REGISTER category:topic key:normal-task subkey:
[nsqllookupd] 2026/06/17 16:17:23.799099 INFO: DB: client(127.0.0.1:34904) REGISTER category:topic key:low-log subkey:
[nsqllookupd] 2026/06/17 16:17:23.799483 INFO: DB: client(127.0.0.1:34904) REGISTER category:channel key:test-topic subkey:demo-channel
[nsqllookupd] 2026/06/17 16:17:23.799419 INFO: DB: client(127.0.0.1:34904) REGISTER category:topic key:test-topic subkey:
[nsqllookupd] 2026/06/17 16:17:23.799714 INFO: DB: client(127.0.0.1:34904) REGISTER category:topic key:high-alert subkey:

@tlinhnguyen205 →/workspaces/nsq-distributed-system (main) $ go run ./apps/nsqd --lookupd-tcp-address=127.0.0.1:4160
[nsqd] 2026/06/17 16:17:23.798618 INFO: LOOKUPD(127.0.0.1:4160): peer info {TCP:4160 HTTP:4161 Version:1.3.0 BroadcastAddress:codespaces-82ba45}
[nsqd] 2026/06/17 16:17:23.798640 INFO: LOOKUPD(127.0.0.1:4160): REGISTER normal-task
[nsqd] 2026/06/17 16:17:23.798933 INFO: LOOKUPD(127.0.0.1:4160): REGISTER low-log
[nsqd] 2026/06/17 16:17:23.799222 INFO: LOOKUPD(127.0.0.1:4160): REGISTER test-topic demo-channel
[nsqd] 2026/06/17 16:17:23.799506 INFO: LOOKUPD(127.0.0.1:4160): REGISTER high-alert
[nsqd] 2026/06/17 16:17:23.799864 INFO: LOOKUPD(127.0.0.1:4160): topic REGISTER normal-task
[nsqd] 2026/06/17 16:17:23.800168 INFO: LOOKUPD(127.0.0.1:4160): topic REGISTER low-log
[nsqd] 2026/06/17 16:17:23.800406 INFO: LOOKUPD(127.0.0.1:4160): channel REGISTER test-topic demo-channel
[nsqd] 2026/06/17 16:17:23.800745 INFO: LOOKUPD(127.0.0.1:4160): topic REGISTER test-topic
[nsqd] 2026/06/17 16:17:23.801005 INFO: LOOKUPD(127.0.0.1:4160): topic REGISTER high-alert

@tlinhnguyen205 →/workspaces/nsq-distributed-system (main) $ go run ./apps/nsqadmin --lookupd-http-address=127.0.0.1:4161
[nsqadmin] 2026/06/17 16:17:32.286179 INFO: nsqadmin v1.3.0 (built w/go1.26.1)
[nsqadmin] 2026/06/17 16:17:32.286652 INFO: HTTP: listening on [::]:4171
```

Hình 3. Các thành phần NSQ hoạt động thành công.

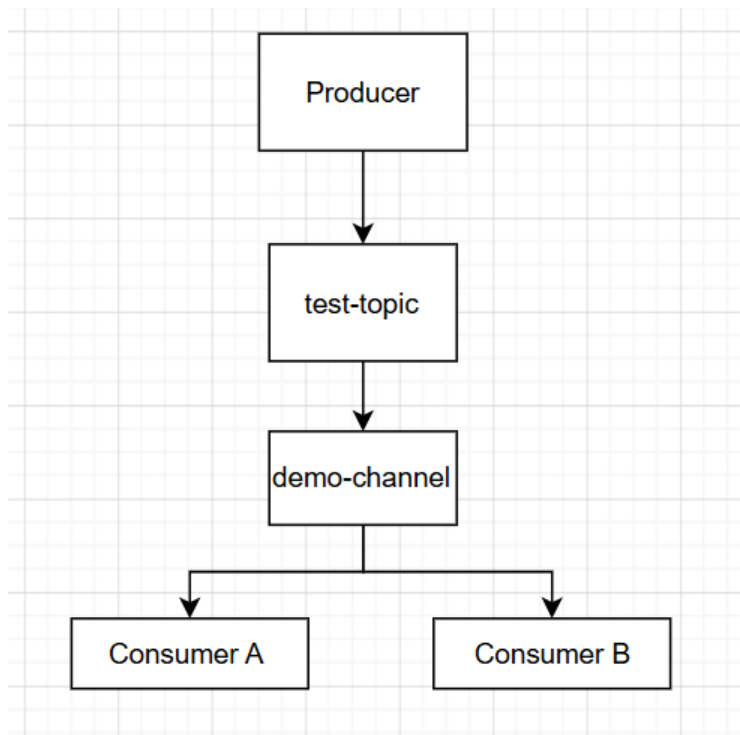
3.4 Thực nghiệm cân bằng tải (Load Balancing)

Nhóm xây dựng mô hình thực nghiệm gồm:

- 01 Producer
- 01 Topic (test-topic)
- 01 Channel (demo-channel)
- 02 Consumer

Hai Consumer cùng đăng ký vào một Channel để kiểm tra cơ chế phân phối thông điệp và cân bằng tải của NSQ.

Mô hình thử nghiệm:



Hình 4. Mô hình thực nghiệm cân bằng tải

Nhóm gửi 100 thông điệp vào Topic test-topic.

Kết quả nhận thông điệp:

- Consumer A nhận một phần các thông điệp được gửi đến hệ thống.
- Consumer B nhận phần thông điệp còn lại.

Client Host	User-Agent	Attributes	NSQd Host	In-Flight	Ready Count	Finished	Requeued	Messages	Uptime
codespaces-82ba45:45706	nsq_to_file/1.3.0 go-nsq/1.1.0		codespaces-82ba45:4151	0	200	49	0	49	2m33s
codespaces-82ba45:45704	nsq_to_file/1.3.0 go-nsq/1.1.0		codespaces-82ba45:4151	0	200	51	0	51	2m41s

Hình 5. Kết quả phân phối thông điệp giữa Consumer A và Consumer B.

Nhận xét:

Kết quả cho thấy các thông điệp được phân phối tự động giữa hai Consumer trong cùng một Channel. Số lượng thông điệp nhận được ở mỗi Consumer có thể không hoàn toàn bằng nhau nhưng đều được xử lý thành công.

Điều này chứng minh cơ chế cân bằng tải (Load Balancing) của NSQ hoạt động hiệu quả và giúp tối ưu tài nguyên xử lý.

3.5 Thực nghiệm khả năng chịu lỗi (Fault Tolerance)

Kịch bản thử nghiệm: Sau khi hệ thống hoạt động ổn định:

- Dừng Consumer A.
- Tiếp tục gửi thêm thông điệp mới.
- Quan sát khả năng xử lý của hệ thống.

Nhóm gửi tiếp 20 thông điệp:

Kết quả:

Client Connections

Client Host	User-Agent	Attributes	NSQd Host	In-Flight	Ready Count	Finished	Queued	Messages	Uptime
codespaces-82ba45:45706	nsq_to_file/1.3.0 go-nsq/1.1.0		codespaces-82ba45:4151	0	200	69	0	69	5m5s

Hình 6. Consumer B nhận toàn bộ thông điệp

Toàn bộ thông điệp từ 101 đến 120 được Consumer B xử lý thành công. Consumer A không còn hoạt động nhưng hệ thống vẫn tiếp tục vận hành bình thường.

Nhận xét:

- Kết quả cho thấy NSQ có khả năng chịu lỗi tốt.
- Khi một Consumer gặp sự cố hoặc ngừng hoạt động, các Consumer còn lại vẫn tiếp tục nhận và xử lý thông điệp mà không làm gián đoạn hệ thống.
- Đây là một đặc điểm quan trọng của hệ thống phân tán hiện đại.

3.6 Đánh giá kết quả thực nghiệm

Qua quá trình cài đặt và thử nghiệm, nhóm nhận thấy:

- NSQ có kiến trúc đơn giản, dễ triển khai.
- Hệ thống hỗ trợ tốt cơ chế Publish/Subscribe.
- Thông điệp được phân phối tự động giữa các Consumer.
- Hệ thống hoạt động ổn định khi xảy ra lỗi Consumer.
- NSQ đáp ứng tốt các đặc tính của hệ thống phân tán gồm:
 - Khả năng mở rộng (Scalability)
 - Cân bằng tải (Load Balancing)
 - Khả năng chịu lỗi (Fault Tolerance)
 - Xử lý bất đồng bộ (Asynchronous Processing)

Kết quả thực nghiệm cho thấy NSQ là một nền tảng truyền thông điệp phân tán hiệu quả và phù hợp với các hệ thống yêu cầu xử lý dữ liệu thời gian thực.

4. PHÁT TRIỂN TÍNH NĂNG MỚI

Sau khi nghiên cứu kiến trúc và thực nghiệm hệ thống NSQ, nhóm tiến hành mở rộng dự án bằng cách phát triển thêm hai tính năng mới liên quan đến khả năng giám sát và quản trị hệ thống phân tán.

Các tính năng được triển khai trực tiếp trên giao diện quản trị NSQAdmin nhằm hỗ trợ người quản trị theo dõi trạng thái hoạt động của hệ thống một cách trực quan hơn.

4.1. Priority-Based Topic Classification and Monitoring

4.1.1. Mục đích

Trong các hệ thống phân tán thực tế, không phải tất cả Topic đều có mức độ quan trọng như nhau.

Ví dụ:

Topic	Mức độ ưu tiên
high-alert	Cao (HIGH)
normal-task	Trung bình (NORMAL)
low-log	Thấp (LOW)

Phiên bản NSQAdmin gốc chỉ hiển thị tên Topic mà không cho biết mức độ ưu tiên của từng Topic. Điều này gây khó khăn cho quản trị viên khi phải theo dõi nhiều Topic cùng lúc.

Do đó nhóm xây dựng tính năng Priority-Based Topic Classification and Monitoring nhằm hỗ trợ phân loại Topic theo mức độ ưu tiên.

4.1.2. Thiết kế

Nhóm chỉnh sửa giao diện danh sách Topic trong NSQAdmin.

Bổ sung thêm cột Priority để hiển thị mức độ ưu tiên của từng Topic.

Điều kiện	Mức ưu tiên
Tên Topic chứa từ khóa "high"	HIGH
Tên Topic chứa từ khóa "normal"	NORMAL
Tên Topic chứa từ khóa "low"	LOW
Các Topic khác	NORMAL

4.1.3. Cài đặt

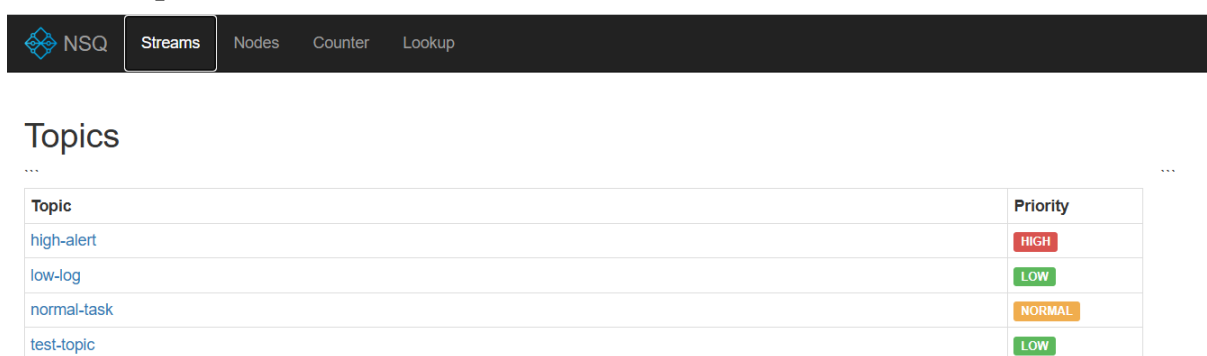
File được chỉnh sửa: nsqadmin/static/js/views/topics.hbs

Nhóm bổ sung cột Priority trong bảng Topic và hiển thị mức độ ưu tiên dưới dạng nhãn màu trực quan.

Ví dụ:

Topic	Priority
high-alert	HIGH
normal-task	NORMAL
low-log	LOW
test-topic	NORMAL

4.1.4. Kết quả



Topic	Priority
high-alert	HIGH
low-log	LOW
normal-task	NORMAL
test-topic	LOW

Hình 7. Giao diện Priority Monitoring

Kết quả cho thấy giao diện NSQAdmin đã hiển thị thêm cột Priority với các mức HIGH, NORMAL và LOW, giúp người quản trị dễ dàng nhận biết các Topic quan trọng trong hệ thống.

4.1.5. Liên hệ với hệ phân tán

Trong các hệ thống phân tán quy mô lớn, số lượng Topic và luồng dữ liệu có thể rất lớn. Việc phân loại Topic theo mức độ ưu tiên giúp quản trị viên nhanh chóng xác định các luồng dữ liệu quan trọng để theo dõi và xử lý.

4.1.6. Đánh giá

Ưu điểm:

- Hỗ trợ quản trị viên nhận biết nhanh Topic quan trọng.
- Giao diện trực quan, dễ sử dụng.
- Dễ dàng mở rộng thành cơ chế ưu tiên xử lý trong tương lai.
- Không ảnh hưởng đến hiệu năng hệ thống.

4.2. Consumer Health Monitoring

4.2.1. Mục đích

Trong hệ thống phân tán, việc theo dõi trạng thái hoạt động của Consumer là rất quan trọng. Một Consumer có thể bị lỗi, mất kết nối hoặc khởi động lại bất thường. Vì vậy nhóm xây dựng tính năng Consumer Health Monitoring nhằm hỗ trợ giám sát tình trạng hoạt động của Consumer.

4.2.2. Thiết kế

Nhóm bổ sung cột Health Status trên giao diện Channel Details.

Hệ thống hiển thị hai trạng thái:

- HEALTHY: Consumer hoạt động ổn định.
- RESTARTED: Consumer vừa khởi động lại hoặc mới kết nối.

Đồng thời hệ thống vẫn hiển thị thời gian hoạt động (Uptime) của Consumer.

4.2.3. Cài đặt

File được chỉnh sửa:

- nsqadmin/static/js/views/channel.hbs
- nsqadmin/static/js/lib/handlebars_helpers.js

Nhóm bổ sung cột Health Status trên giao diện Channel Details để hiển thị trạng thái hoạt động của Consumer.

Hệ thống sử dụng thuộc tính `{{nanotohuman connected}}` để chuyển đổi thời gian kết nối của Consumer sang định dạng dễ đọc và hiển thị dưới dạng Uptime.

Ngoài ra, nhóm xây dựng helper `if_consumer_restarted` nhằm đánh giá trạng thái của Consumer dựa trên thời gian kết nối. Nếu Consumer vừa kết nối hoặc vừa khởi động lại, hệ thống hiển thị trạng thái RESTARTED; ngược lại sẽ hiển thị trạng thái HEALTHY.

Thông tin Health Status và Uptime được hiển thị trực tiếp trên giao diện NSQAdmin để hỗ trợ giám sát hoạt động của Consumer.

4.2.4. Kết quả

NSQd Host	In-Flight	Ready Count	Finished	Requeued	Messages	Health Status	Uptime
codespaces-82ba45:4151	0	200	119	0	119	HEALTHY	1h48m4s
codespaces-82ba45:4151	0	200	50	0	50	HEALTHY	1h6m45s

Hình 8. Giao diện Consumer Health Monitoring

Kết quả thực nghiệm: Kết quả thực nghiệm cho thấy hệ thống hiển thị đồng thời trạng thái Health Status và thời gian hoạt động (Uptime) của từng Consumer. Người quản trị có thể dễ dàng nhận biết các Consumer đang hoạt động ổn định thông qua trạng thái HEALTHY hoặc phát hiện các Consumer vừa được khởi động lại thông qua trạng thái RESTARTED.

4.2.5. Kiểm thử

STT	Trường hợp kiểm thử	Kết quả mong đợi	Kết quả thực tế	Trạng thái
1	Consumer đang hoạt động bình thường	Health Status = HEALTHY và Uptime tăng liên tục theo thời gian	Health Status hiển thị HEALTHY, Uptime tăng liên tục	PASS
2	Consumer bị dừng và khởi động lại	Health Status chuyển sang RESTARTED và Uptime được đặt lại từ 0	Health Status chuyển sang RESTARTED, Uptime được đặt lại từ thời điểm kết nối mới	PASS
3	Nhiều Consumer hoạt động đồng thời	Mỗi Consumer có Health Status và Uptime riêng	Mỗi Consumer hiển thị Health Status và Uptime riêng biệt	PASS

4.2.6. Liên hệ với hệ phân tán

Trong môi trường phân tán, nhiều Consumer có thể hoạt động trên các node khác nhau. Việc theo dõi thời gian hoạt động giúp phát hiện sớm các Consumer không ổn định hoặc thường xuyên khởi động lại, từ đó hỗ trợ nâng cao độ tin cậy của hệ thống

4.2.7. Đánh giá

Ưu điểm:

- Hỗ trợ giám sát tính ổn định của Consumer.
- Giúp phát hiện Consumer khởi động lại bất thường.
- Hỗ trợ vận hành và bảo trì hệ thống.
- Cung cấp thông tin trực quan cho quản trị viên trong quá trình giám sát hệ thống.

4.3 Tổng kết các tính năng mở rộng

STT	Tính năng	Mục đích
-----	-----------	----------

1	Priority-Based Topic Classification and Monitoring	Phân loại và giám sát Topic theo mức độ ưu tiên nhằm hỗ trợ quản trị viên nhanh chóng xác định các luồng thông điệp quan trọng trong hệ thống phân tán.
2	Consumer Health Monitoring	Theo dõi trạng thái hoạt động và thời gian kết nối của Consumer, hỗ trợ giám sát các thành phần xử lý thông điệp, phát hiện Consumer vừa khởi động lại hoặc có dấu hiệu hoạt động không ổn định.

Bảng 4.1. Tổng hợp tính năng mới

Hai tính năng trên đã được triển khai thành công trên hệ thống NSQAdmin và hoạt động ổn định trong quá trình thực nghiệm. Các tính năng giúp nâng cao khả năng giám sát, quản trị và vận hành hệ thống phân tán so với phiên bản NSQAdmin ban đầu.

5. ĐÓNG GÓP CỦA THÀNH VIÊN

5.1 Phân công công việc

Trong quá trình thực hiện đề tài, các thành viên trong nhóm đã phối hợp nghiên cứu, cài đặt, thực nghiệm và phát triển tính năng mới cho hệ thống NSQ. Việc phân công công việc được thực hiện dựa trên khả năng và thời gian tham gia của từng thành viên.

Thành viên	Công việc thực hiện
Nguyễn Thị Thuỳ Linh	Tìm hiểu kiến trúc hệ thống NSQ, nghiên cứu tài liệu dự án, thiết lập môi trường GitHub Codespaces, cài đặt và cấu hình NSQ, thực hiện thực nghiệm Load Balancing và Fault Tolerance, phát triển tính năng Priority-Based Topic Classification and Monitoring, tổng hợp kết quả thực nghiệm, viết báo cáo và chuẩn bị slide thuyết trình
Nguyễn Anh Quân	Nghiên cứu cơ chế hoạt động của Producer, Consumer, Topic và Channel, hỗ trợ thực nghiệm hệ thống, kiểm tra kết quả truyền thông điệp, phát triển tính năng Consumer Health Monitoring, hỗ trợ chỉnh sửa mã nguồn, tham gia hoàn thiện báo cáo và slide

5.2 Kết quả làm việc nhóm

Nhóm đã hoàn thành đầy đủ các yêu cầu của bài tập giữa kỳ bao gồm:

- Tìm hiểu và nghiên cứu dự án NSQ.
- Cài đặt thành công hệ thống trên GitHub Codespaces.
- Thực nghiệm cơ chế Publish/Subscribe.
- Thực nghiệm cân bằng tải giữa nhiều Consumer.
- Thực nghiệm khả năng chịu lỗi của hệ thống.
- Phát triển thành công hai tính năng mới:
 - Priority-Based Topic Classification and Monitoring.
 - Consumer Health Monitoring
- Quản lý mã nguồn trên GitHub.
- Hoàn thiện báo cáo và slide trình bày.

Thông qua quá trình thực hiện đề tài, các thành viên đã hiểu rõ hơn về kiến trúc hệ thống phân tán, cơ chế truyền thông điệp, khả năng mở rộng và chịu lỗi của các nền tảng Distributed Messaging hiện đại.

KẾT LUẬN

Trong đề tài này, nhóm đã nghiên cứu, cài đặt và thực nghiệm thành công hệ thống truyền thông điệp phân tán NSQ. Kết quả thực nghiệm cho thấy NSQ đáp ứng tốt các yêu cầu của một hệ thống phân tán hiện đại như xử lý bất đồng bộ, cân bằng tải giữa nhiều Consumer và khả năng chịu lỗi khi xảy ra sự cố trong quá trình vận hành.

Bên cạnh việc tìm hiểu kiến trúc và cơ chế hoạt động của NSQ, nhóm đã triển khai thành công hai tính năng mở rộng gồm Priority-Based Topic Classification and Monitoring và Consumer Health Monitoring trên giao diện NSQAdmin. Các tính năng này góp phần nâng cao khả năng giám sát và quản trị hệ thống, đồng thời thể hiện khả năng mở rộng và tùy biến của dự án mã nguồn mở.

Thông qua quá trình thực hiện đề tài, nhóm đã củng cố kiến thức về hệ phân tán, cơ chế Publish/Subscribe, Service Discovery, Load Balancing, Fault Tolerance và các phương pháp giám sát hệ thống phân tán. Đây là những kiến thức quan trọng có thể được áp dụng trong việc xây dựng các hệ thống Microservices, hệ thống xử lý dữ liệu thời gian thực và các ứng dụng quy mô lớn trong thực tế.

Trong tương lai, các tính năng đã phát triển có thể được mở rộng theo hướng xây dựng cơ chế ưu tiên xử lý thông điệp, cảnh báo khi Consumer hoạt động bất thường và hỗ trợ giám sát hệ thống theo thời gian thực nhằm nâng cao hiệu quả vận hành của nền tảng NSQ.